Artificial Intelligence Project #2
111550132 張家睿

1. Research Question and Motivation

Recently, self-supervised learning (SSL) has become highly popular because it can learn feature representations without relying on human-annotated labels. In this project, we focus on SimCLR, a contrastive learning method that trains the model to pull augmented views of the same image closer together in the representation space.

The main objective of this project is to implement and evaluate the SimCLR framework. Specifically, we aim to explore the following questions:

(1) How does the performance of SimCLR on CIFAR-10 compare to a standard supervised learning model trained from scratch?

(2) How do different hyperparameters, especially the temperature scaling and the use of a projector head, affect the final representation quality?

(3) When transferring the learned representations to a new dataset, does the SSL model generalize better than the supervised baseline?

2. Methods

2.1 Model Architecture

We use a modified ResNet-18 as the backbone encoder. To accommodate 32×32 CIFAR-10 images, the first convolutional layer is changed from 7×7 (stride=2) to 3×3 (stride=1, padding=1), and the subsequent max-pooling layer is replaced with an identity operation. The backbone outputs a 512-dimensional feature vector after global average pooling. A two-layer MLP projector head (512→512→128 with ReLU) maps features to a 128-dim space for contrastive loss computation. All models are trained from scratch (no pretrained weights).

2.2 SimCLR Training

For each batch of N images, two independent augmented views are generated, producing 2N samples. Augmentations include: RandomResizedCrop (scale 0.2–1.0), RandomHorizontalFlip, ColorJitter (brightness/contrast/saturation, hue), RandomGrayscale, and normalization with CIFAR-10 statistics. The NT-Xent loss is computed on L2-normalized projector outputs: for each view, the loss is the negative log-softmax probability of its mate among 2N−1 candidates, scaled by temperature τ. Training uses Adam optimizer (lr=3e-4, weight decay=1e-6) for 200 epochs with batch size 256.

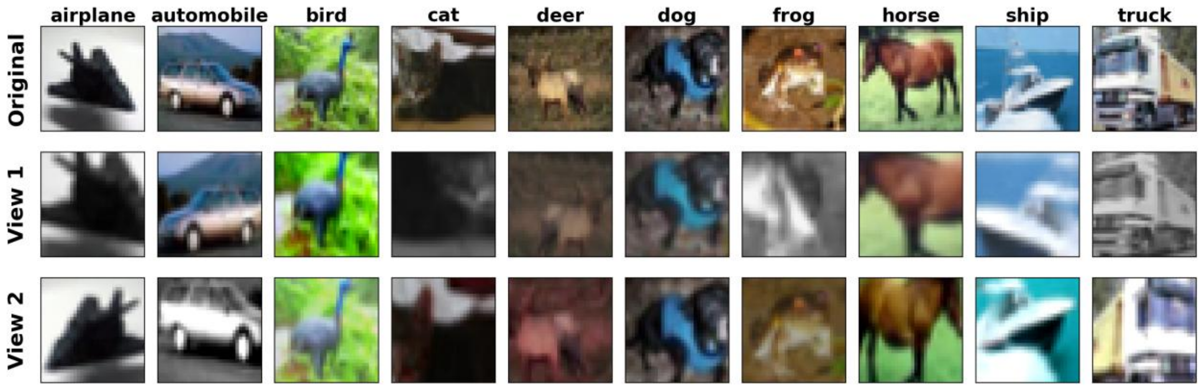

Figure 1: SimCLR augmentation examples, original image (left) and two independently augmented views (middle, right) for each CIFAR-10 class.

Figure 1 shows augmentation examples for all 10 CIFAR-10 classes. Each row displays the original image and two independently sampled views. The diversity of crops, color shifts, and grayscale conversions forces the model to learn content-invariant features rather than relying on color or spatial cues.

2.3 Evaluation Methods

KNN Monitor: Every 5 epochs, we compute k-nearest-neighbor classification (k=20) on the test set using L2-normalized backbone features of the training set as the memory bank. This provides a training-free measure of representation quality.

Linear Probing: After SSL training, we freeze the backbone and train a single linear layer (512→num_classes) with Adam for 100 epochs. This evaluates the linear separability of learned representations.

Supervised Baseline: The same backbone architecture with a classification head (512→10) is trained end-to-end with cross-entropy loss, Adam, and standard augmentation (RandomCrop with padding=4, RandomHorizontalFlip) for 200 epochs.

2.4 Tools and Libraries

Implementation uses PyTorch and torchvision. The modified ResNet-18 is based on torchvision.models.resnet18 with surgical modifications. Code structure is modularized into config.py, dataset.py, model.py, utils.py, train_simclr.py, train_supervised.py, and linear_eval.py.

3. Experiments and Results

3.1 SimCLR Baseline

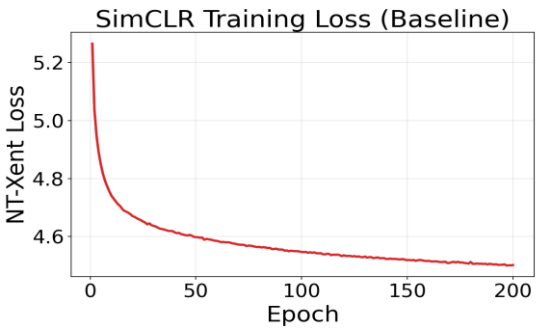

Figure 3a: NT-Xent Loss

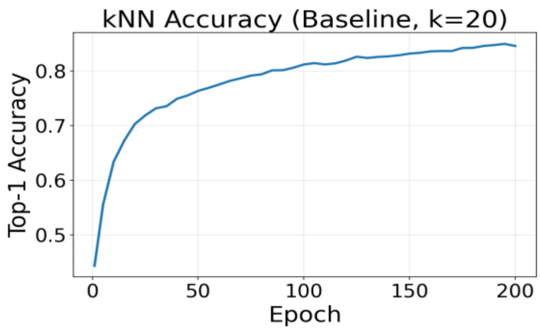

Figure 3b: KNN Accuracy (k=20)

The NT-Xent loss decreases steadily from 5.265 to 4.502 over 200 epochs (Fig. 2a). The KNN accuracy rises rapidly in early training reaching 63.3% by epoch 10 and continues improving to 84.55% at epoch 200, though gains slow after epoch 100 (Fig. 2b).

Epoch	KNN Accuracy
1	44.31%
10	63.33%
50	76.33%
100	81.16%
150	83.15%
200	84.55%

3.2 Linear Probing on SSL Model

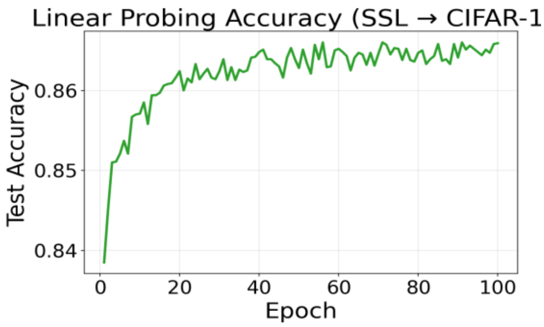

Figure 3: SSL Linear Probing Accuracy on CIFAR-10

Freezing the SimCLR backbone and training a linear classifier for 100 epochs yields a final test accuracy of 86.60%, confirming that the learned representations are linearly separable across CIFAR-10 classes.

3.3 Supervised Baseline

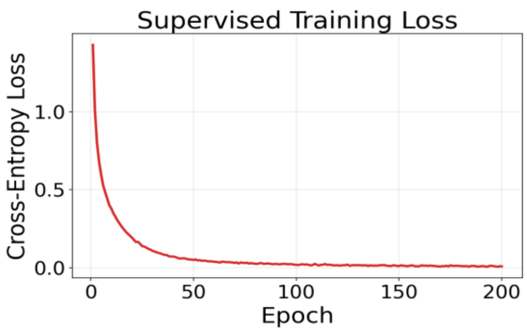

Figure 6a: Supervised Loss

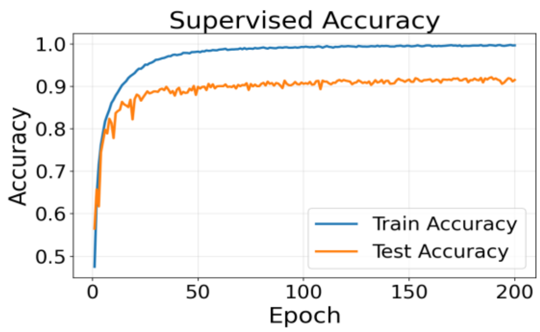

Figure 6b: Supervised Accuracy

The supervised model reaches 99.65% train accuracy but 91.51% test accuracy (best: 92.16% at epoch 189), showing a ~8% generalization gap indicative of overfitting.

3.4 SSL vs. Supervised vs. Random Baseline

Method	CIFAR-10 Test Acc	Note
Supervised Learning	91.51%	End-to-end with labels
SimCLR + Linear Probe	86.60%	No labels for backbone
SimCLR KNN (Epoch 200)	84.55%	Training-free evaluation
Random Backbone + LP	40.29%	Lower bound

SimCLR linear probing (86.60%) trails supervised learning (91.51%) by only 4.91 percentage points, despite using zero labels for backbone training. The random baseline (40.29%) confirms that the SSL training provides substantial representation quality: To quantify: the improvement of SSL over random is $86.60 - 40.29 = 46.31\%$, while the improvement of supervised over random is $91.51 - 40.29 = 51.22\%$. Thus, SSL recovers $46.31 / 51.22 \approx 90.4\%$).

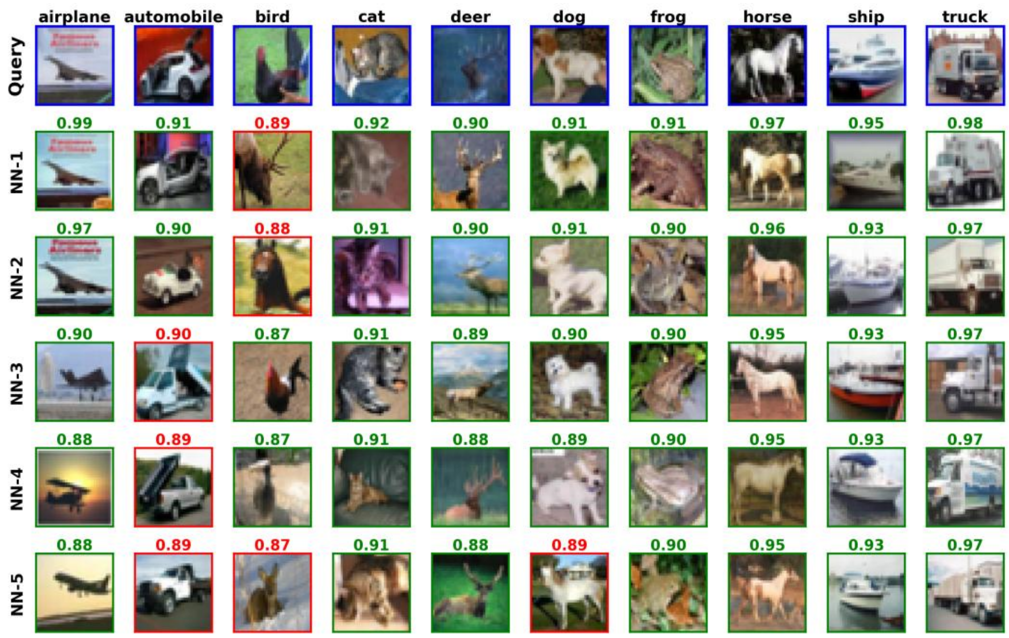

Figure 5: KNN retrieval examples, for each query image (blue border, left), the 5 nearest neighbors from the training set are shown. Green borders indicate correct class matches; red borders indicate mismatches. Cosine similarity scores are displayed above each neighbor.

Figure 5 provides qualitative evidence of representation quality. For most classes, all 5 nearest neighbors share the same class as the query, and similarity scores are high (>0.85). Mismatches tend to occur between visually similar classes, revealing interpretable failure modes that reflect genuine visual similarity rather than random errors.

3.5 Temperature Ablation

We train SimCLR with three temperature values: τ=0.1 (low), τ=0.5 (baseline), and τ=5.0 (high). All other settings are identical.

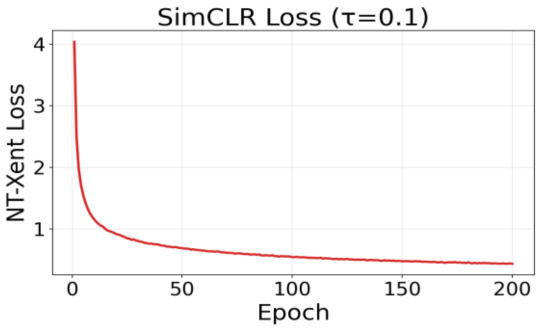

Figure 7a: Loss (τ=0.1)

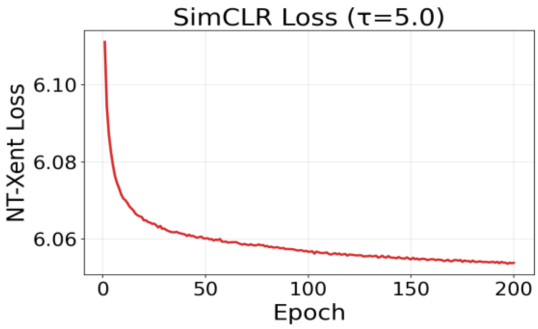

Figure 7b: Loss (τ=5.0)

Temperature	Initial Loss	Final Loss	Loss Drop
0.1	4.036	0.433	3.603
0.5 (baseline)	5.265	4.502	0.763
5.0	6.111	6.054	0.057

The loss magnitude is not comparable across temperatures, τ directly scales the logits, so τ=0.1 produces the largest drop while τ=5.0 barely moves. This demonstrates that the loss value alone is a poor indicator of representation quality in SSL.

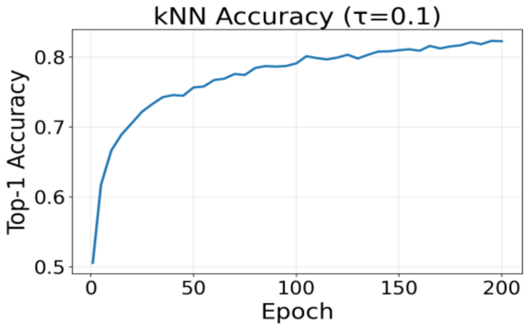

Figure 8a: KNN (τ=0.1)

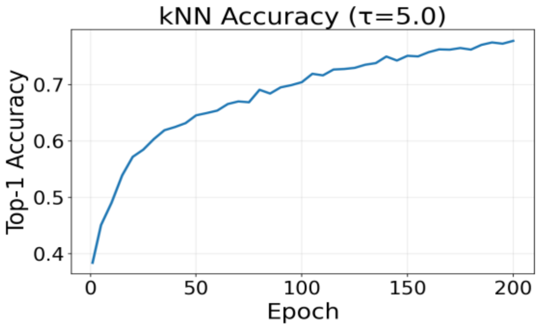

Figure 8b: KNN (τ=5.0)

Epoch	τ=0.1	τ=0.5 (baseline)	τ=5.0
1	50.59%	44.31%	38.36%
50	75.65%	76.33%	64.57%
100	79.09%	81.16%	70.46%
200	82.26%	84.55%	77.80%

Temperature	KNN (Epoch 200)	Linear Probing
0.1	82.26%	84.43%
0.5 (baseline)	84.55%	86.60%
5.0	77.80%	82.03%

τ=0.1 learns faster initially (50.59% vs 44.31% at epoch 1) but plateaus lower, because the overly peaked softmax causes the model to focus on the hardest negatives, producing less globally structured representations. τ=5.0 learns very slowly, as the flattened softmax provides weak gradient signals. τ=0.5 offers the best balance, achieving the highest KNN (84.55%) and linear probing (86.60%).

3.6 Projector Head Ablation

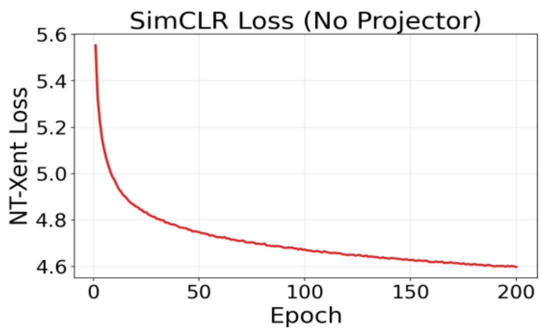

Figure 7a: No Projector, Loss

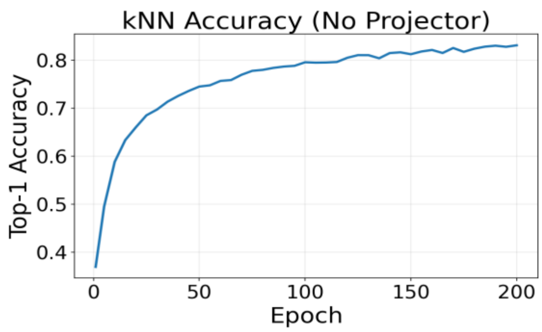

Figure 7b: No Projector, KNN

Model	KNN (Epoch 200)	Linear Probing
With Projector (baseline)	84.55%	86.60%
Without Projector	83.15%	83.96%
Difference	−1.40%	−2.64%

Removing the projector head decreases linear probing accuracy by 2.64%. The projector acts as an information bottleneck that absorbs task-specific distortions from the contrastive loss, allowing the backbone to retain more general-purpose features. Without it, the NT-Xent loss directly shapes the backbone's 512-dim output, over-specializing it for the contrastive task at the expense of downstream linear separability.

3.7 Transfer Learning to CIFAR-100

We freeze backbones trained on CIFAR-10 (SSL, Supervised, Random) and perform linear probing on CIFAR-100 (100 classes, 500 images/class). Images are resized to 32×32 and normalized with CIFAR-100 statistics. Linear probing settings are identical to CIFAR-10.

Backbone	CIFAR-10 LP	CIFAR-100 LP
SimCLR (SSL)	86.60%	50.50%
Supervised (SL)	91.51%	49.48%
Random	40.29%	19.16%

A striking result: SSL (50.50%) surpasses Supervised (49.48%) on CIFAR-100 transfer, despite trailing by 4.91% on CIFAR-10. This reversal demonstrates that SSL's instance discrimination objective learns more transferable features than supervised learning, which overfits to the specific 10-class label space of CIFAR-10. The relative accuracy drop from CIFAR-10 to CIFAR-100 is also smaller for SSL (41.7%) than for Supervised (45.9%), further confirming the generalization advantage.

4. Discussion

4.1 Based on your experiments, are the results and observed behaviors what you expect?

Before running the experiments, I expected the supervised model to clearly outperform SSL on CIFAR-10, since it has direct access to the labels. The actual gap turned out to be about 5% (91.51% vs. 86.60%), which is smaller than I initially thought. Looking at the original SimCLR paper afterward, this seems to be roughly in line with what they report for ResNet-18 at moderate batch sizes, so the result is reasonable, though I suspect our batch size of 256 is on the lower end of what SimCLR really needs to shine.

One thing out of my expectations was how fast the KNN accuracy climbed in the first few epochs. By epoch 10, the model was already at 63%, which suggests the backbone picks up coarse-grained class structure quite early and then spends the remaining epochs refining finer distinctions.

The temperature results were less surprising, since $\tau = 0.5$ is already in the commonly recommended range. But I would not have been able to predict how much $\tau = 5.0$ would hurt; losing nearly 7 percentage points compared to the baseline was more dramatic than I expected. The $\tau = 0.1$ case was interesting in a different way: it actually starts faster but ends up worse. I wouldn't have thought of this based on the reading theory section.

The projector head ablation was consistent with what the SimCLR paper reports, and the 2.64% drop without it felt about right.

4.2 Discuss factors that affect the results, including dataset characteristics.

1. Batch size: SimCLR relies on in-batch negatives. With batch size 256, the model sees 511 negative pairs per sample. Larger batches would likely improve performance.
2. Augmentation: The combination of crop, color jitter, and grayscale is critical. Crop forces spatial invariance; color jitter forces color invariance.
3. Training duration: The KNN curve at epoch 200 has not fully saturated, suggesting longer training could further improve results. Dataset scale: CIFAR-10 has only 50,000 32×32 images. SimCLR benefits more from larger datasets (like ImageNet) where supervised labeling is expensive.

4.3 Describe experiments that you would do if there were more time available.

1. Batch size ablation to quantify the effect of negative sample diversity.
2. Augmentation ablation to identify the most critical transforms.
3. Longer training epochs to determine if the SSL-SL gap continues to narrow.
4. Using projector outputs (128-dim) as representations during evaluation.

5. Comparing with other SSL methods such as SimSiam.

4.4 Indicate what you have learned from the experiments as well as your remaining questions.

The most important takeaway is that the contrastive loss value is not a reliable training signal, KNN monitoring is essential for tracking actual representation quality. The temperature ablation vividly illustrates this: τ=0.1 produces the largest loss drop but not the best representations. Second, the projector head is not just an architectural detail but a principled design choice that protects the backbone's representation space. Third, the SSL transfer advantage on CIFAR-100 demonstrates why self-supervised learning is valuable as a foundation model approach: the representations are not tied to a specific label set and generalize better to new tasks.

My remaining questions

1. The KNN accuracy at epoch 200 (84.55%) has not fully saturated. How much further could it improve with 500 or 1000 epochs? Is there a point of diminishing returns, or could the SSL-SL gap be closed entirely with sufficient training?
2. The projector head ablation showed a 2.64% drop without it. But does a deeper or wider projector yield further gains, or is the current 2-layer MLP already near optimal for this scale?

5. References

[1] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," ICML 2020. arXiv:2002.05709.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR 2016.

[3] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009.

[4] PyTorch: https://pytorch.org/

[5] torchvision: https://pytorch.org/vision/

[6] Claude Code was used as an AI coding assistant for implementation.

Appendix: Program Code

(Not counting toward the 10-page limit)

config.py

```
"""Central configuration for all experiments."""

import argparse
from dataclasses import dataclass, field, fields
from typing import Optional


@dataclass
class Config:
    # --- Data ---
    dataset: str = "cifar10"
    data_dir: str = "./data"
    num_workers: int = 2
    image_size: int = 32

    # --- Model ---
    backbone: str = "resnet18"
    feature_dim: int = 512
    projector_hidden_dim: int = 512
    projector_out_dim: int = 128
    use_projector: bool = True  # set False to ablate projector head

    # --- SimCLR Training ---
    batch_size: int = 256
    ssl_epochs: int = 200
    ssl_lr: float = 3e-4
    ssl_weight_decay: float = 1e-6
    temperature: float = 0.5

    # --- KNN Monitor ---
    KNN_k: int = 20
    KNN_every: int = 5

    # --- Linear Probing ---
    lp_epochs: int = 100
    lp_lr: float = 1e-3
    lp_weight_decay: float = 1e-6
    lp_dataset: str = "cifar10"  # "cifar10", "cifar100", "stl10"

    # --- Supervised Training ---
    sl_epochs: int = 200
    sl_lr: float = 3e-4
    sl_weight_decay: float = 1e-6
    num_classes: int = 10

    # --- System ---
    device: str = "cuda"
    seed: int = 42
    checkpoint_dir: str = "./checkpoints"
    log_dir: str = "./logs"
    resume: Optional[str] = None  # path to checkpoint, or "random"

    # --- Experiment Tag ---
    experiment_name: str = "baseline"


def get_config() -> Config:
    """Parse command-line arguments and return a Config object.
```

```
    Every field in Config can be overridden from the CLI, e.g.:
        python train_simclr.py --batch_size 512 --temperature 0.1
    """
    config = Config()
    parser = argparse.ArgumentParser()

    for f in fields(Config):
        ftype = f.type
        # Handle Optional[str] — check string representation as fallback
        if ftype is Optional[str] or str(ftype) in ("typing.Optional[str]",
                                                     "str | None"):
            ftype = str
        if ftype is bool:
            parser.add_argument(
                f"--{f.name}",
                type=lambda v: v.lower() in ("true", "1", "yes"),
                default=getattr(config, f.name),
            )
        else:
            parser.add_argument(
                f"--{f.name}", type=ftype, default=getattr(config, f.name)
            )

    args = parser.parse_args()
    return Config(**vars(args))
```

dataset.py

```
"""Data loading and augmentation pipelines for SimCLR, supervised, and linear probing."""

from typing import Tuple

import torchvision
import torchvision.transforms as T
from torch.utils.data import DataLoader

from config import Config

# ---------- Normalization stats ----------

CIFAR10_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR10_STD = (0.2023, 0.1994, 0.2010)

CIFAR100_MEAN = (0.5071, 0.4867, 0.4408)
CIFAR100_STD = (0.2675, 0.2565, 0.2761)

STL10_MEAN = (0.4408, 0.4279, 0.3867)
STL10_STD = (0.2682, 0.2610, 0.2686)

IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)


def _get_normalize(dataset: str):
    stats = {
        "cifar10": (CIFAR10_MEAN, CIFAR10_STD),
        "cifar100": (CIFAR100_MEAN, CIFAR100_STD),
        "stl10": (STL10_MEAN, STL10_STD),
    }
    mean, std = stats.get(dataset, (IMAGENET_MEAN, IMAGENET_STD))
    return T.Normalize(mean=mean, std=std)
```

```python
# ---------- Augmentations ----------

class SimCLRAugmentation:
    """Returns two independently augmented views of the same image."""

    def __init__(self, image_size: int = 32, dataset: str = "cifar10"):
        self.transform = T.Compose([
            T.RandomResizedCrop(size=image_size, scale=(0.2, 1.0)),
            T.RandomHorizontalFlip(p=0.5),
            T.RandomApply([
                T.ColorJitter(brightness=0.4, contrast=0.4,
                              saturation=0.4, hue=0.1)
            ], p=0.8),
            T.RandomGrayscale(p=0.2),
            T.ToTensor(),
            _get_normalize(dataset),
        ])

    def __call__(self, x):
        return self.transform(x), self.transform(x)


def _get_test_transform(image_size: int = 32, dataset: str = "cifar10"):
    """Deterministic transform for evaluation (no augmentation)."""
    ops = []
    # Resize for datasets with images larger than 32x32
    if dataset in ("stl10",):
        ops.append(T.Resize(image_size))
    ops.append(T.ToTensor())
    ops.append(_get_normalize(dataset))
    return T.Compose(ops)


def _get_supervised_train_transform(image_size: int = 32, dataset: str = "cifar10"):
    """Standard supervised training augmentation."""
    return T.Compose([
        T.RandomCrop(image_size, padding=4),
        T.RandomHorizontalFlip(p=0.5),
        T.ToTensor(),
        _get_normalize(dataset),
    ])


# ---------- Dataset factories ----------

def _get_num_classes(dataset: str) -> int:
    return {"cifar10": 10, "cifar100": 100, "stl10": 10}.get(dataset, 10)


def _load_dataset(name: str, root: str, train: bool, transform):
    """Load a torchvision dataset by name."""
    if name == "cifar10":
        return torchvision.datasets.CIFAR10(
            root=root, train=train, transform=transform, download=True
        )
    elif name == "cifar100":
        return torchvision.datasets.CIFAR100(
            root=root, train=train, transform=transform, download=True
        )
    elif name == "stl10":
        split = "train" if train else "test"
        return torchvision.datasets.STL10(
            root=root, split=split, transform=transform, download=True
        )
```

```python
    else:
        raise ValueError(f"Unsupported dataset: {name}")


# ---------- DataLoader factories ----------

def get_simclr_loaders(
    config: Config,
) -> Tuple[DataLoader, DataLoader, DataLoader]:
    """Return (ssl_train_loader, memory_loader, test_loader).

    - ssl_train_loader: training set with SimCLR dual augmentation.
    - memory_loader:    training set with test transform (for KNN memory bank).
    - test_loader:      test set with test transform.
    """
    ssl_transform = SimCLRAugmentation(config.image_size, config.dataset)
    test_transform = _get_test_transform(config.image_size, config.dataset)

    train_ssl = _load_dataset(config.dataset, config.data_dir, train=True,
                              transform=ssl_transform)
    train_mem = _load_dataset(config.dataset, config.data_dir, train=True,
                              transform=test_transform)
    test_set = _load_dataset(config.dataset, config.data_dir, train=False,
                             transform=test_transform)

    ssl_train_loader = DataLoader(
        train_ssl, batch_size=config.batch_size, shuffle=True,
        num_workers=config.num_workers, pin_memory=True, drop_last=True,
    )
    memory_loader = DataLoader(
        train_mem, batch_size=config.batch_size, shuffle=False,
        num_workers=config.num_workers, pin_memory=True,
    )
    test_loader = DataLoader(
        test_set, batch_size=config.batch_size, shuffle=False,
        num_workers=config.num_workers, pin_memory=True,
    )
    return ssl_train_loader, memory_loader, test_loader


def get_supervised_loaders(
    config: Config,
) -> Tuple[DataLoader, DataLoader]:
    """Return (train_loader, test_loader) for supervised training."""
    train_transform = _get_supervised_train_transform(
        config.image_size, config.dataset
    )
    test_transform = _get_test_transform(config.image_size, config.dataset)

    train_set = _load_dataset(config.dataset, config.data_dir, train=True,
                              transform=train_transform)
    test_set = _load_dataset(config.dataset, config.data_dir, train=False,
                             transform=test_transform)

    train_loader = DataLoader(
        train_set, batch_size=config.batch_size, shuffle=True,
        num_workers=config.num_workers, pin_memory=True, drop_last=True,
    )
    test_loader = DataLoader(
        test_set, batch_size=config.batch_size, shuffle=False,
        num_workers=config.num_workers, pin_memory=True,
    )
    return train_loader, test_loader
```

```python
def get_linear_eval_loaders(
    config: Config,
) -> Tuple[DataLoader, DataLoader]:
    """Return (train_loader, test_loader) for linear probing.

    Uses ``config.lp_dataset`` which may differ from the SSL training dataset.
    """
    dataset_name = config.lp_dataset
    test_transform = _get_test_transform(config.image_size, dataset_name)

    # Linear probing uses deterministic transform (no augmentation) for both
    train_set = _load_dataset(dataset_name, config.data_dir, train=True,
                              transform=test_transform)
    test_set = _load_dataset(dataset_name, config.data_dir, train=False,
                             transform=test_transform)

    train_loader = DataLoader(
        train_set, batch_size=config.batch_size, shuffle=True,
        num_workers=config.num_workers, pin_memory=True,
    )
    test_loader = DataLoader(
        test_set, batch_size=config.batch_size, shuffle=False,
        num_workers=config.num_workers, pin_memory=True,
    )
    return train_loader, test_loader
```

model.py

```python
"""Network architectures: modified ResNet-18, projector head, and combined models."""

import torch
import torch.nn as nn
import torchvision.models as models

from config import Config


# ---------- Backbone ----------

def get_backbone(config: Config) -> nn.Module:
    """Create a modified ResNet-18 suitable for CIFAR-10 (32x32 inputs).

    Modifications from the standard torchvision ResNet-18:
      1. conv1: 3x3, stride=1, padding=1 (was 7x7, stride=2, padding=3)
      2. maxpool after conv1 replaced with nn.Identity()
      3. Final fc layer replaced with nn.Identity() -> output is 512-dim
    All weights are randomly initialised (no pretrained weights).
    """
    backbone = models.resnet18(weights=None)
    # Smaller first convolution for 32x32 images
    backbone.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
    # Remove aggressive downsampling
    backbone.maxpool = nn.Identity()
    # Remove classification head; keep avgpool + flatten -> 512-dim
    backbone.fc = nn.Identity()
    return backbone


# ---------- Projector Head ----------

class ProjectorHead(nn.Module):
    """Two-layer MLP projector: input_dim -> hidden_dim -> output_dim."""
```

```python
    def __init__(self, input_dim: int = 512, hidden_dim: int = 512,
                 output_dim: int = 128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(inplace=True),
            nn.Linear(hidden_dim, output_dim),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


# ---------- Combined Models ----------

class SimCLRModel(nn.Module):
    """Backbone + projector for SimCLR self-supervised training.

    forward() returns (features, projections):
      - features:    backbone output (512-dim), used for KNN monitor
      - projections: projector output (128-dim), used for NT-Xent loss
    """

    def __init__(self, config: Config):
        super().__init__()
        self.backbone = get_backbone(config)
        if config.use_projector:
            self.projector = ProjectorHead(
                config.feature_dim,
                config.projector_hidden_dim,
                config.projector_out_dim,
            )
        else:
            self.projector = nn.Identity()

    def forward(self, x: torch.Tensor):
        features = self.backbone(x)           # (B, 512)
        projections = self.projector(features) # (B, 128) or (B, 512)
        return features, projections


class SupervisedModel(nn.Module):
    """Backbone + linear classification head for supervised training."""

    def __init__(self, config: Config):
        super().__init__()
        self.backbone = get_backbone(config)
        self.classifier = nn.Linear(config.feature_dim, config.num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        features = self.backbone(x)
        return self.classifier(features)


class LinearClassifier(nn.Module):
    """Single linear layer for linear probing evaluation."""

    def __init__(self, feature_dim: int = 512, num_classes: int = 10):
        super().__init__()
        self.fc = nn.Linear(feature_dim, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```python
        return self.fc(x)
```

utils.py

```python
"""Shared utilities: NT-Xent loss, KNN monitor, checkpointing, metrics logging."""

import os
import random
from collections import defaultdict

import matplotlib
matplotlib.use("Agg")  # non-interactive backend
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader


# ========================================================================
# NT-Xent Loss
# ========================================================================

class NTXentLoss(nn.Module):
    """Normalized Temperature-scaled Cross-Entropy Loss (NT-Xent).

    Expects input ``z`` of shape (2N, D) where z[i] and z[i+N] are mates
    (two augmented views of the same source image).  ``z`` should be
    L2-normalised before being passed in.
    """

    def __init__(self, temperature: float = 0.5):
        super().__init__()
        self.temperature = temperature

    def forward(self, z: torch.Tensor) -> torch.Tensor:
        """Compute NT-Xent loss.

        Args:
            z: (2N, D) L2-normalised projection vectors.

        Returns:
            Scalar loss averaged over all 2N views.
        """
        device = z.device
        batch_size = z.shape[0] // 2  # N

        # Pairwise cosine similarity (z is unit-norm -> dot product)
        sim = z @ z.T / self.temperature  # (2N, 2N)

        # Mask out self-similarity on the diagonal
        mask = torch.eye(2 * batch_size, dtype=torch.bool, device=device)
        sim = sim.masked_fill(mask, -1e9)

        # Positive-pair labels: view i <-> view i+N
        labels = torch.cat([
            torch.arange(batch_size, 2 * batch_size, device=device),
            torch.arange(0, batch_size, device=device),
        ])  # (2N,)

        loss = F.cross_entropy(sim, labels)
        return loss
```

```python
# ==========================================================================
# KNN Monitor
# ==========================================================================

@torch.no_grad()
def KNN_evaluate(
    backbone: nn.Module,
    memory_loader: DataLoader,
    test_loader: DataLoader,
    k: int = 20,
    num_classes: int = 10,
    device: str = "cuda",
) -> float:
    """k-Nearest-Neighbour classification accuracy on the test set.

    1. Extract L2-normalised features for the full training set (memory bank).
    2. For each test image, find k nearest neighbours and do weighted voting.
    3. Return top-1 accuracy.
    """
    backbone.eval()

    # Build memory bank from training set
    feature_bank, label_bank = [], []
    for images, labels in memory_loader:
        features = backbone(images.to(device))
        features = F.normalize(features, dim=1)
        feature_bank.append(features)
        label_bank.append(labels.to(device))

    feature_bank = torch.cat(feature_bank, dim=0)  # (M, 512)
    label_bank = torch.cat(label_bank, dim=0)       # (M,)

    # Evaluate on test set
    correct, total = 0, 0
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)

        features = backbone(images)
        features = F.normalize(features, dim=1)

        # Cosine similarity with memory bank
        sim = features @ feature_bank.T  # (B, M)
        topk_sim, topk_idx = sim.topk(k, dim=1)  # (B, k)
        topk_labels = label_bank[topk_idx]  # (B, k)

        # Weighted voting: accumulate similarity for each class
        votes = torch.zeros(images.size(0), num_classes, device=device)
        votes.scatter_add_(1, topk_labels, topk_sim)

        preds = votes.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    return correct / total


# ==========================================================================
# Seed
# ==========================================================================

def set_seed(seed: int):
```

```python
    """Set random seed for reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)


# ==========================================================================
# Checkpointing
# ==========================================================================

def save_checkpoint(state: dict, filepath: str):
    """Save a checkpoint dictionary to disk."""
    os.makedirs(os.path.dirname(filepath), exist_ok=True)
    torch.save(state, filepath)
    print(f"  -> Checkpoint saved: {filepath}")


def load_checkpoint(filepath: str, device: str = "cuda") -> dict:
    """Load a checkpoint dictionary from disk."""
    state = torch.load(filepath, map_location=device, weights_only=False)
    print(f"  -> Checkpoint loaded: {filepath}")
    return state


# ==========================================================================
# Metrics Logger
# ==========================================================================

class MetricsLogger:
    """Accumulate per-epoch metrics and produce plots / CSV."""

    def __init__(self):
        self.history: dict[str, list] = defaultdict(list)
        self.epochs: dict[str, list] = defaultdict(list)

    def log(self, epoch: int, **kwargs):
        """Log one or more named metrics for the given epoch."""
        for name, value in kwargs.items():
            self.history[name].append(value)
            self.epochs[name].append(epoch)

    def save_plot(self, metrics: list[str], filepath: str, title: str = "",
                  ylabel: str = ""):
        """Plot specified metrics vs. epoch and save to file."""
        os.makedirs(os.path.dirname(filepath) or ".", exist_ok=True)
        fig, ax = plt.subplots(figsize=(8, 5))
        for m in metrics:
            ax.plot(self.epochs[m], self.history[m], label=m)
        ax.set_xlabel("Epoch")
        ax.set_ylabel(ylabel or ", ".join(metrics))
        ax.set_title(title or ", ".join(metrics))
        ax.legend()
        ax.grid(True, alpha=0.3)
        fig.tight_layout()
        fig.savefig(filepath, dpi=150)
        plt.close(fig)
        print(f"  -> Plot saved: {filepath}")

    def save_csv(self, filepath: str):
        """Export all metrics to CSV."""
        os.makedirs(os.path.dirname(filepath) or ".", exist_ok=True)
        all_metrics = sorted(self.history.keys())
        # Find the union of all epochs
```

```
        all_epochs = sorted(set(e for ep_list in self.epochs.values()
                                for e in ep_list))
        # Build lookup: metric -> {epoch: value}
        lookup = {}
        for m in all_metrics:
            lookup[m] = dict(zip(self.epochs[m], self.history[m]))

        with open(filepath, "w") as f:
            f.write("epoch," + ",".join(all_metrics) + "\n")
            for ep in all_epochs:
                vals = [str(lookup[m].get(ep, "")) for m in all_metrics]
                f.write(f"{ep}," + ",".join(vals) + "\n")
        print(f"  -> CSV saved: {filepath}")
```

train_simclr.py

```
"""SimCLR self-supervised training with KNN monitoring.

Usage:
    python train_simclr.py                              # baseline
    python train_simclr.py --temperature 0.1 --experiment_name temp01
    python train_simclr.py --batch_size 512 --experiment_name bs512
    python train_simclr.py --use_projector False --experiment_name no_proj
"""

import os
import time

import torch
import torch.nn.functional as F
from tqdm import tqdm

from config import get_config
from dataset import get_simclr_loaders, _get_num_classes
from model import SimCLRModel
from utils import (
    NTXentLoss,
    MetricsLogger,
    KNN_evaluate,
    save_checkpoint,
    set_seed,
)


def train_one_epoch(model, loader, optimizer, criterion, device):
    """Run one epoch of SimCLR training. Returns average loss."""
    model.train()
    total_loss = 0.0

    for (x1, x2), _ in tqdm(loader, desc="  train", leave=False):
        # Concatenate two views: (N,C,H,W) x 2 -> (2N,C,H,W)
        x = torch.cat([x1, x2], dim=0).to(device)

        # Forward pass
        _features, projections = model(x)  # (2N, 512), (2N, 128)
        z = F.normalize(projections, dim=1)

        loss = criterion(z)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```
        total_loss += loss.item()

    return total_loss / len(loader)


def main():
    config = get_config()
    set_seed(config.seed)
    print(f"=== SimCLR Training: {config.experiment_name} ===")
    print(f"    batch_size={config.batch_size}  temperature={config.temperature}  "
          f"epochs={config.ssl_epochs}  use_projector={config.use_projector}")

    # Data
    ssl_loader, memory_loader, test_loader = get_simclr_loaders(config)
    num_classes = _get_num_classes(config.dataset)

    # Model
    model = SimCLRModel(config).to(config.device)
    optimizer = torch.optim.Adam(
        model.parameters(), lr=config.ssl_lr, weight_decay=config.ssl_weight_decay
    )
    criterion = NTXentLoss(temperature=config.temperature)
    logger = MetricsLogger()

    start_epoch = 1
    # Resume from checkpoint if specified
    if config.resume and config.resume != "random":
        ckpt = torch.load(config.resume, map_location=config.device, weights_only=False)
        model.load_state_dict(ckpt["model_state"])
        optimizer.load_state_dict(ckpt["optimizer_state"])
        start_epoch = ckpt["epoch"] + 1
        print(f"  Resumed from epoch {ckpt['epoch']}")

    # Training loop
    for epoch in range(start_epoch, config.ssl_epochs + 1):
        t0 = time.time()
        train_loss = train_one_epoch(
            model, ssl_loader, optimizer, criterion, config.device
        )
        elapsed = time.time() - t0
        logger.log(epoch, ssl_loss=train_loss)

        # KNN monitor
        KNN_str = ""
        if epoch % config.KNN_every == 0 or epoch == 1:
            KNN_acc = KNN_evaluate(
                model.backbone, memory_loader, test_loader,
                k=config.KNN_k, num_classes=num_classes, device=config.device,
            )
            logger.log(epoch, KNN_accuracy=KNN_acc)
            KNN_str = f"  KNN={KNN_acc:.2%}"

        print(f"Epoch [{epoch}/{config.ssl_epochs}]  "
              f"Loss={train_loss:.4f}{KNN_str}  ({elapsed:.1f}s)")

        # Checkpoint
        if epoch % 50 == 0 or epoch == config.ssl_epochs:
            ckpt_path = os.path.join(
                config.checkpoint_dir,
                f"{config.experiment_name}_epoch{epoch}.pt",
            )
            save_checkpoint({
                "epoch": epoch,
                "model_state": model.state_dict(),
```

```
                "backbone_state": model.backbone.state_dict(),
                "optimizer_state": optimizer.state_dict(),
                "config": vars(config),
            }, ckpt_path)

    # Save plots and CSV
    log_dir = os.path.join(config.log_dir, config.experiment_name)
    os.makedirs(log_dir, exist_ok=True)

    logger.save_plot(
        ["ssl_loss"],
        os.path.join(log_dir, "loss_curve.png"),
        title=f"SimCLR Loss ({config.experiment_name})",
        ylabel="NT-Xent Loss",
    )
    logger.save_plot(
        ["KNN_accuracy"],
        os.path.join(log_dir, "KNN_curve.png"),
        title=f"KNN Accuracy ({config.experiment_name})",
        ylabel="Top-1 Accuracy",
    )
    logger.save_csv(os.path.join(log_dir, "metrics.csv"))

    print("Done.")


if __name__ == "__main__":
    main()
```

train_supervised.py

```
"""Supervised baseline training from scratch.

Usage:
    python train_supervised.py --batch_size 256 --experiment_name supervised
"""

import os
import time

import torch
import torch.nn as nn
from tqdm import tqdm

from config import get_config
from dataset import get_supervised_loaders, _get_num_classes
from model import SupervisedModel
from utils import MetricsLogger, save_checkpoint, set_seed


def train_one_epoch(model, loader, optimizer, criterion, device):
    """One epoch of supervised training. Returns (avg_loss, train_accuracy)."""
    model.train()
    total_loss = 0.0
    correct, total = 0, 0

    for images, labels in tqdm(loader, desc="  train", leave=False):
        images, labels = images.to(device), labels.to(device)

        logits = model(images)
        loss = criterion(logits, labels)

        optimizer.zero_grad()
```

```python
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        preds = logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    return total_loss / len(loader), correct / total


@torch.no_grad()
def evaluate(model, loader, device):
    """Evaluate classification accuracy on a dataset."""
    model.eval()
    correct, total = 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        preds = logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    return correct / total


def main():
    config = get_config()
    set_seed(config.seed)
    config.num_classes = _get_num_classes(config.dataset)
    print(f"=== Supervised Training: {config.experiment_name} ===")
    print(f"    batch_size={config.batch_size}  epochs={config.sl_epochs}  "
          f"num_classes={config.num_classes}")

    # Data
    train_loader, test_loader = get_supervised_loaders(config)

    # Model
    model = SupervisedModel(config).to(config.device)
    optimizer = torch.optim.Adam(
        model.parameters(), lr=config.sl_lr, weight_decay=config.sl_weight_decay
    )
    criterion = nn.CrossEntropyLoss()
    logger = MetricsLogger()

    # Training loop
    for epoch in range(1, config.sl_epochs + 1):
        t0 = time.time()
        train_loss, train_acc = train_one_epoch(
            model, train_loader, optimizer, criterion, config.device
        )
        test_acc = evaluate(model, test_loader, config.device)
        elapsed = time.time() - t0

        logger.log(epoch, sl_loss=train_loss, sl_train_acc=train_acc,
                   sl_test_acc=test_acc)

        print(f"Epoch [{epoch}/{config.sl_epochs}]  "
              f"Loss={train_loss:.4f}  TrainAcc={train_acc:.2%}  "
              f"TestAcc={test_acc:.2%}  ({elapsed:.1f}s)")

        # Checkpoint
        if epoch % 50 == 0 or epoch == config.sl_epochs:
```

```
            ckpt_path = os.path.join(
                config.checkpoint_dir,
                f"{config.experiment_name}_epoch{epoch}.pt",
            )
            save_checkpoint({
                "epoch": epoch,
                "model_state": model.state_dict(),
                "backbone_state": model.backbone.state_dict(),
                "optimizer_state": optimizer.state_dict(),
                "config": vars(config),
            }, ckpt_path)

    # Save plots and CSV
    log_dir = os.path.join(config.log_dir, config.experiment_name)
    os.makedirs(log_dir, exist_ok=True)

    logger.save_plot(
        ["sl_loss"],
        os.path.join(log_dir, "loss_curve.png"),
        title=f"Supervised Loss ({config.experiment_name})",
        ylabel="Cross-Entropy Loss",
    )
    logger.save_plot(
        ["sl_train_acc", "sl_test_acc"],
        os.path.join(log_dir, "accuracy_curve.png"),
        title=f"Supervised Accuracy ({config.experiment_name})",
        ylabel="Accuracy",
    )
    logger.save_csv(os.path.join(log_dir, "metrics.csv"))

    print(f"Final test accuracy: {test_acc:.2%}")
    print("Done.")


if __name__ == "__main__":
    main()
```

linear_eval.py

```
"""Linear probing evaluation.

Freezes a pretrained backbone and trains a single linear layer for classification.

Usage:
    # Evaluate SSL model
    python linear_eval.py --resume checkpoints/baseline_epoch200.pt --experiment_name ssl_probe

    # Evaluate supervised model
    python linear_eval.py --resume checkpoints/supervised_epoch200.pt --experiment_name sl_probe

    # Random baseline (no training, just random weights)
    python linear_eval.py --resume random --experiment_name random_probe

    # Transfer to CIFAR-100
    python linear_eval.py --resume checkpoints/baseline_epoch200.pt \
        --lp_dataset cifar100 --experiment_name ssl_cifar100
"""

import os
import time

import torch
import torch.nn as nn
```

```python
from tqdm import tqdm

from config import get_config
from dataset import get_linear_eval_loaders, _get_num_classes
from model import LinearClassifier, get_backbone
from utils import MetricsLogger, save_checkpoint, set_seed, load_checkpoint


@torch.no_grad()
def evaluate(backbone, classifier, loader, device):
    """Evaluate accuracy with frozen backbone + linear classifier."""
    backbone.eval()
    classifier.eval()
    correct, total = 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        features = backbone(images)
        logits = classifier(features)
        preds = logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    return correct / total


def main():
    config = get_config()
    set_seed(config.seed)

    lp_dataset = config.lp_dataset
    num_classes = _get_num_classes(lp_dataset)

    print(f"=== Linear Probing: {config.experiment_name} ===")
    print(f"    backbone from: {config.resume}")
    print(f"    eval dataset: {lp_dataset}  num_classes={num_classes}")
    print(f"    epochs={config.lp_epochs}  lr={config.lp_lr}")

    # Load backbone
    backbone = get_backbone(config).to(config.device)

    if config.resume is None or config.resume == "random":
        print("  Using randomly initialised backbone (lower-bound baseline).")
    else:
        ckpt = load_checkpoint(config.resume, config.device)
        backbone.load_state_dict(ckpt["backbone_state"])
        print("  Backbone weights loaded successfully.")

    # Freeze backbone
    backbone.eval()
    for param in backbone.parameters():
        param.requires_grad = False

    # Data
    train_loader, test_loader = get_linear_eval_loaders(config)

    # Linear classifier
    classifier = LinearClassifier(config.feature_dim, num_classes).to(config.device)
    optimizer = torch.optim.Adam(
        classifier.parameters(), lr=config.lp_lr, weight_decay=config.lp_weight_decay
    )
    criterion = nn.CrossEntropyLoss()
    logger = MetricsLogger()
```

```python
    # Training loop
    best_acc = 0.0
    for epoch in range(1, config.lp_epochs + 1):
        t0 = time.time()

        # Train linear layer
        classifier.train()
        total_loss = 0.0
        for images, labels in tqdm(train_loader, desc="  train", leave=False):
            images, labels = images.to(config.device), labels.to(config.device)
            with torch.no_grad():
                features = backbone(images)
            logits = classifier(features)
            loss = criterion(logits, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        avg_loss = total_loss / len(train_loader)
        test_acc = evaluate(backbone, classifier, test_loader, config.device)
        elapsed = time.time() - t0
        best_acc = max(best_acc, test_acc)

        logger.log(epoch, lp_loss=avg_loss, lp_test_acc=test_acc)

        if epoch % 10 == 0 or epoch == config.lp_epochs:
            print(f"Epoch [{epoch}/{config.lp_epochs}]  "
                  f"Loss={avg_loss:.4f}  TestAcc={test_acc:.2%}  ({elapsed:.1f}s)")

    # Save plots and CSV
    log_dir = os.path.join(config.log_dir, config.experiment_name)
    os.makedirs(log_dir, exist_ok=True)

    logger.save_plot(
        ["lp_test_acc"],
        os.path.join(log_dir, "linear_probe_acc.png"),
        title=f"Linear Probing ({config.experiment_name})",
        ylabel="Test Accuracy",
    )
    logger.save_csv(os.path.join(log_dir, "metrics.csv"))

    # Save final classifier
    ckpt_path = os.path.join(
        config.checkpoint_dir, f"{config.experiment_name}_linear.pt"
    )
    save_checkpoint({
        "classifier_state": classifier.state_dict(),
        "best_acc": best_acc,
        "config": vars(config),
    }, ckpt_path)

    print(f"\nFinal test accuracy: {test_acc:.2%}  (best: {best_acc:.2%})")
    print("Done.")


if __name__ == "__main__":
    main()
```